

AI Fraud-Spike Detector

Technical Specification v1.0

Canonical Project Specification

20 August 2026

Status: Canonical implementation specification
Project type: Defense-only AI/ML fraud-risk monitoring system
Primary use case: E-commerce/payment transaction fraud-spike detection
Primary dataset: IEEE-CIS Fraud Detection
Primary evaluation mode: Chronological out-of-time replay on a held-out test period

Contents

1. Purpose
 2. Core Problem Definition
 3. Scope
 4. Dataset Strategy
 5. Data Preparation
 6. Initial Feature Priorities
 7. Missing-Value Strategy
 8. Chronological Split and Validation Protocol
 9. Transaction-Risk Layer
 10. Definition of Suspicious Activity
 11. Time Aggregation Layer
 12. Baseline Estimation
 13. Spike-Detection Layer
 14. Ground Truth for Fraud Spikes
 15. Alert Decision and Severity
 16. Cost Framework
 17. Evaluation Framework
 18. Near-Real-Time Replay Architecture
 19. End-to-End System Architecture
 20. Dashboard and Alerting Requirements
 21. Modeling Experiments
 22. Threshold Selection
 23. Reproducibility Requirements
 24. Acceptance Criteria
 25. Key Risks and Limitations
 26. Locked Decisions vs Data-Driven Decisions
 27. Final System Definition
 28. Implementation Sequence
 29. Dataset Verification References
 30. Version Notes
-

1. Purpose

This document defines the technical specification for the **AI Fraud-Spike Detector**.

The system is designed to identify **statistically and operationally meaningful increases in suspicious transaction activity over time**. It is not intended to declare every suspicious transaction fraudulent, and it is not an offensive fraud tool.

The specification separates the problem into two layers:

1. **Transaction-risk layer** — estimates the probability that each incoming transaction is fraudulent.
2. **Fraud-spike layer** — aggregates those transaction-risk signals over time, estimates normal behavior, detects abnormal increases, and generates merchant-facing alerts.

The system must be evaluated using a held-out chronological test period, with explicit treatment of precision, recall, false positives, detection latency, and modeled business cost.

2. Core Problem Definition

A merchant may process thousands of transactions while experiencing a non-zero amount of fraud as part of normal operations. Therefore, the existence of individual suspicious transactions does not by itself imply that a fraud event is occurring.

The system must answer two different questions:

2.1 Transaction-level question

Is this individual transaction suspicious enough to deserve elevated risk treatment?

The answer is a model-generated **fraud probability / risk score**, not a fraud verdict.

2.2 Spike-level question

Is suspicious transaction activity currently increasing beyond the level that is normally expected?

The spike detector observes the behavior of many transaction-level risk signals across time windows.

2.3 Important distinctions

- **Fraud is not the same as anomaly.**
 - **A risk score is not certainty.**
 - **Suspicious activity is not confirmed fraud.**
 - **Normal does not mean zero fraud.**
 - **A spike is not an arbitrary fixed percentage.**
 - **Ground-truth transaction labels may only become known after investigation.**
-

3. Scope

3.1 In scope

- E-commerce/payment transaction fraud risk.
- Historical labeled transaction modeling.
- Transaction-level fraud probability estimation.

- Time-based aggregation of predicted risk.
- Rolling baseline estimation.
- Statistical/change-based spike detection.
- Near-real-time simulation through chronological historical replay.
- Cost-aware threshold selection.
- Merchant-facing anomaly alerts and dashboard.
- Held-out out-of-time evaluation.

3.2 Out of scope for v1

- Offensive fraud generation or evasion.
 - Automatic accusation of customers.
 - Automatic permanent blocking based solely on the spike detector.
 - Chargeback evidence generation.
 - Return-risk prediction.
 - Abuse-ring graph detection as a primary objective.
 - Production payment-gateway integration.
 - Production-scale streaming infrastructure.
 - Final merchant-specific rupee cost values without real merchant data.
 - A deep-learning model solely for complexity or novelty.
-

4. Dataset Strategy

4.1 Primary dataset

The project will use the **IEEE-CIS Fraud Detection** dataset.

The labeled competition training data contains transaction records and a separate identity table. These are joined using **TransactionID**. Not every transaction has a corresponding identity record.

The transaction table includes groups such as:

- TransactionID
- TransactionDT
- TransactionAmt
- ProductCD
- card1-card6
- addr1, addr2
- distance-related features
- purchaser/recipient email-domain features
- C1-C14
- D1-D15
- M1-M9
- V1-V339
- target isFraud

The identity table includes:

- TransactionID
- id_01-id_38
- DeviceType
- DeviceInfo

4.2 Important time-field constraint

TransactionDT is a **time delta from an undisclosed reference point**, not a real calendar timestamp.

Therefore:

- The project will preserve chronological ordering using TransactionDT.
- A “1-hour window” means a **3,600-second interval in relative dataset time**.
- The project must not claim that a window corresponds to a specific real-world clock hour unless the hidden reference timestamp becomes available.
- Derived features should use terms such as `relative_hour`, `elapsed_day`, or `time_window_id`, rather than pretending the dataset exposes real local clock time.

4.3 Label availability

The labeled training data contains `isFraud`.

For supervised transaction-risk modeling:

- `isFraud = 1` is treated as the positive fraud label.
- `isFraud = 0` is treated as the non-fraud label.
- `isFraud` is **never used as an input feature**.
- It is used only for training/evaluation and for offline construction of spike ground truth.

4.4 Dataset limitations

The IEEE-CIS dataset is suitable for this prototype, but the specification acknowledges:

- Many feature meanings are anonymized.
 - Identity data is not present for every transaction.
 - Missingness can be substantial.
 - The dataset provides transaction fraud labels, not explicit “fraud attack” or “fraud spike” incident labels.
 - Competition labels are treated as ground truth for the experiment, not as proof that real-world fraud labeling is perfect.
 - The labeled competition training set is reused as the project’s train/validation/test source; the unlabeled Kaggle competition test set is not used for final model scoring.
-

5. Data Preparation

5.1 Join strategy

1. Load labeled transaction data.
2. Load labeled identity data.
3. Left-join identity onto transactions using TransactionID.

4. Preserve all transaction rows.
5. Retain TransactionID for traceability and debugging.
6. Do not use TransactionID as a predictive feature.

5.2 Ordering

Before any time-dependent split or rolling feature computation:

sort by TransactionDT ascending

If two or more transactions share the same TransactionDT, the project should avoid using future rows from the same timestamp to construct past-only features.

5.3 Data-type audit

EDA must classify columns as:

- identifier
- target
- numeric
- categorical
- relative time
- masked engineered feature

No feature is automatically accepted simply because it exists in the source dataset.

5.4 Leakage audit

Before modeling, every candidate feature must be checked for:

- direct target leakage
- future information
- whole-dataset statistics
- future entity counts
- future rolling values
- transformations fitted using validation/test data

Any feature unavailable at prediction time must be removed or reconstructed in a past-only manner.

6. Initial Feature Priorities

The final feature list will be EDA- and validation-driven. The initial feature families are prioritized as follows.

6.1 Priority A — transaction amount

Primary raw feature:

- TransactionAmt

Potential derived features:

- log-transformed amount, if useful
- amount relative to past entity history

- amount deviation from rolling median/mean
- high-value indicator based on training-derived thresholds

6.2 Priority B — relative time

Primary raw feature:

- TransactionDT

Derived features may include:

- relative 3,600-second window ID
- elapsed day index
- time since previous transaction for a selected entity
- rolling transaction velocity
- rolling suspicious-risk velocity

The project must not label a relative-time bucket as a true “hour of day” without a known reference timestamp.

6.3 Priority C — product/card features

Initial candidates include:

- ProductCD
- card1-card6

Potential behavioral features:

- transaction count per card/entity in prior windows
- average amount per card/entity from past data only
- number of distinct observed devices per card from past data only
- amount deviation from past card/entity behavior

6.4 Priority D — identity/device features

Initial candidates include:

- DeviceType
- DeviceInfo
- id_01-id_38, subject to EDA, missingness, type, and validation value

Potential derived features:

- transaction count per device
- number of distinct cards associated with a device
- device/card velocity
- missing-identity indicators

6.5 Priority E — address and supporting transaction features

Initial candidates include:

- addr1, addr2
- distance features where available
- email-domain features

- C1-C14
- D1-D15
- M1-M9
- selected V1-V339 features

These features are candidates, not guaranteed final inputs.

6.6 Core engineered behavioral features

The first engineering pass should investigate:

- txn_count_entity_past_window
- amount_mean_entity_past_window
- amount_median_entity_past_window
- amount_deviation_from_entity_history
- unique_cards_per_device_past
- unique_devices_per_card_past
- transaction_velocity
- mean_predicted_risk_window
- high_risk_count_window
- suspicious_rate_window
- high_risk_amount_window

All entity/history features must be computed using information available **strictly before the prediction time**.

7. Missing-Value Strategy

Missingness is treated as both a data-quality issue and a potentially informative signal.

7.1 EDA requirements

For every candidate feature, calculate:

- missing count
- missing percentage
- missingness by fraud label
- missingness by time segment
- missingness by major product/device groups where useful

7.2 Numerical variables

Default v1 strategy:

Median imputation fitted on the training data only.

Rationale:

- Robust to skewed distributions and outliers.

- Reproducible in a preprocessing pipeline.

7.3 Categorical variables

Default v1 strategy:

Replace missing category values with an explicit *Unknown* category.

Do not automatically replace missing categories with the most frequent category.

7.4 Missingness indicators

For selected features where missingness itself may carry predictive information, add binary indicators such as:

DeviceInfo_missing = 1

The indicator list must be selected using training/validation evidence.

7.5 Extremely sparse features

A feature with very high missingness is not automatically dropped or retained.

Decision process:

1. quantify missingness;
2. assess potential information value;
3. test validation contribution;
4. drop if unreliable, unstable, or non-contributory.

The project will not use a fabricated universal missingness cutoff.

7.6 Train-only fitting

All imputation values and learned preprocessing parameters must be fitted on the appropriate training portion only.

The validation/test period must never influence:

- medians
 - category statistics
 - encoding mappings that depend on frequency
 - feature-selection decisions
 - scaler statistics if scaling is later required
-

8. Chronological Split and Validation Protocol

8.1 No random split

The dataset must **never be shuffled before the outer split**.

Reason:

The deployment simulation requires:

past -> predict future

not:

random past/future mixture -> predict another random mixture

8.2 Outer split

After sorting the labeled IEEE-CIS training data by TransactionDT:

- **Earliest 70%:** training period
- **Next 15%:** validation period
- **Latest 15%:** final held-out test period

Using the commonly reported 590,540 labeled transaction rows, the nominal row counts are:

Split	Nominal count	Sorted row-index range
Train	413,378	0–413,377
Validation	88,581	413,378–501,958
Test	88,581	501,959–590,539

These are nominal boundaries. If a boundary falls inside a group of identical TransactionDT values, move the boundary so the same timestamp is not split across two partitions.

8.3 Expanding-window cross-validation inside training

Model hyperparameters are selected using expanding chronological folds.

Nominal folds over the full ordered dataset:

Fold 1: train 0–30% -> validate 30–40%

Fold 2: train 0–40% -> validate 40–50%

Fold 3: train 0–50% -> validate 50–60%

Fold 4: train 0–60% -> validate 60–70%

No validation fold may contain transactions earlier than any transaction in its corresponding training fold.

8.4 Role of the 15% validation period

After transaction-model hyperparameters are selected, the next 15% is used to select/tune:

- transaction suspicious-risk threshold
- time-window choice
- rolling-baseline length
- z-score / rolling-statistic thresholds
- EWMA parameters
- CUSUM parameters
- severity levels
- spike alert threshold
- cost-sensitive operating point
- optional Isolation Forest comparison settings
- proxy spike-ground-truth construction parameters, without using test model outputs

8.5 Final test-period rules

The latest 15% remains untouched until final evaluation.

During chronological replay:

- each transaction is processed in time order;
 - no future feature values may be used;
 - no future labels may be used for prediction;
 - no later window may influence the current window;
 - baseline history may use train + validation and previous test observations only;
 - final labels are revealed only for offline scoring.
-

9. Transaction-Risk Layer

9.1 Primary model

The primary v1 transaction-risk model is:

XGBoost binary classifier

Output:

$P(\text{isFraud} = 1 \mid \text{information available at transaction time})$

9.2 Baseline comparator

A simpler supervised classifier such as logistic regression should be retained as a benchmark where the preprocessing makes that comparison practical.

The purpose is to demonstrate whether XGBoost adds meaningful value over a simpler baseline.

9.3 Class imbalance

The dataset is strongly imbalanced. Therefore:

- overall accuracy is not a primary success metric;
- threshold-independent and threshold-dependent metrics must both be reported;
- class weights / scale-pos-weight may be explored only using training/validation data;
- resampling, if tested, must occur strictly inside training folds.

9.4 Output preservation

For each transaction, retain both:

fraud_probability

suspicious_flag

The continuous score is required even if a binary threshold is used.

10. Definition of Suspicious Activity

10.1 Transaction-level suspicious flag

A transaction becomes “suspicious” when:

$\text{fraud_probability} \geq \text{tau_tx}$

where tau_tx is the transaction-risk decision threshold.

10.2 Threshold selection

tau_tx is **not fixed at 0.5**.

It must be selected using validation data based on:

- precision
- recall
- alert volume
- false-positive cost
- missed-fraud cost
- overall expected cost / benefit

10.3 Continuous-risk aggregation

The spike layer must retain the option to use:

1. binary suspicious-transaction count/rate;
2. average/sum of continuous fraud probabilities;
3. high-risk transaction count;
4. monetary exposure associated with high-risk transactions.

This allows the project to test whether continuous risk provides a better spike signal than hard thresholding alone.

11. Time Aggregation Layer

11.1 Initial window

The initial aggregation window is:

3,600 seconds of relative TransactionDT time.

This is called a **1-hour relative window**, not a confirmed real-world clock hour.

11.2 Experimental window sizes

Validation experiments may compare:

- 900 seconds (15 minutes)
- 1,800 seconds (30 minutes)
- 3,600 seconds (1 hour)
- 21,600 seconds (6 hours)

The final v1 window is selected on validation based on detection quality, stability, and latency.

11.3 Per-window features

For each window, compute at minimum:

- transaction_count
- suspicious_count
- suspicious_rate
- mean_fraud_probability
- sum_fraud_probability
- high_risk_count
- total_transaction_amount
- high_risk_amount

Optional contextual features may include:

- product/card mix shifts
- device mix shifts
- amount distribution changes
- identity-missingness shifts

Only features supported by the source data and past-only processing may be used.

12. Baseline Estimation

12.1 Primary baseline

The primary baseline is a **rolling historical baseline** computed from preceding time windows.

Initial components:

- rolling mean of suspicious rate / risk signal
- rolling standard deviation
- standardized deviation score

Conceptually:

$$z_t = (\text{current_signal}_t - \text{rolling_mean_past}) / \text{rolling_std_past}$$

The current window must not be included in the statistics used to define its own expected baseline.

12.2 Baseline history length

The exact number of preceding windows is selected on the validation period.

The specification does not invent a fixed “24-hour” baseline without testing because the dataset uses relative time and transaction volume may vary.

12.3 Warm start

At the beginning of the held-out test period, baseline calculations may use chronologically earlier train/validation history.

This is realistic because a deployed system would already have prior observations.

13. Spike-Detection Layer

13.1 Primary statistical detector

The first spike detector is:

Rolling baseline + standardized deviation (z-score/rolling-statistic alert rule).

This is the transparent benchmark.

13.2 Comparative statistical detectors

The project will compare the primary baseline against:

- **EWMA**
- **CUSUM**

These methods are appropriate for detecting sustained or accumulating shifts that may not appear as a single extreme window.

13.3 Experimental ML anomaly baseline

Isolation Forest may be included as an experimental unsupervised comparator on aggregated window-level features.

It is not the primary model unless validation/test evidence demonstrates a clear benefit.

13.4 Advanced models

Autoencoders, temporal neural networks, transformers, graph models, and Bayesian change-point models are not required for v1.

They may be explored only if the simpler methods reveal a specific limitation that justifies additional complexity.

14. Ground Truth for Fraud Spikes

14.1 Why a proxy definition is required

IEEE-CIS supplies **transaction fraud labels**, not explicit incident-level fraud-spike annotations.

Therefore, spike-level precision and recall cannot be honestly reported until an offline operational definition of a true spike is created.

14.2 Ground-truth signal

For each eligible aggregation window in validation/test, compute using final offline labels:

actual_fraud_rate =
confirmed_fraud_transactions / total_transactions

These labels are used **only after the fact for evaluation**, never to generate the live detector signal.

14.3 Eligibility rule

Very low-volume windows can generate unstable percentages.

Therefore, a window is eligible for spike-ground-truth labeling only when its transaction count is at or above a minimum-volume threshold learned from the training distribution.

Default procedure:

use a training-derived lower-volume quantile and confirm the choice on validation.

The test set does not choose this threshold.

14.4 Primary spike-label rule

Because no incident labels exist, the project will use a validation-derived high-tail threshold on **actual fraud rate** as the primary proxy definition.

Default v1 rule:

$\text{tau_GT} = 97.5\text{th percentile}$
of eligible validation-window actual_fraud_rate

A test window is a proxy true-spike window if:

$\text{actual_fraud_rate_test} > \text{tau_GT}$

The threshold is derived from validation labels only and is independent of test predictions.

14.5 Event merging

Consecutive positive proxy-spike windows are merged into one spike event.

Event onset is the first positive window.

14.6 Sensitivity analysis

Because the spike label is a proxy rather than an externally confirmed incident label, final reporting must include sensitivity at:

- 95th percentile
- 97.5th percentile (primary)
- 99th percentile

This prevents the project from presenting one arbitrary tail threshold as universal truth.

14.7 Synthetic stress testing

A separate stress-test experiment may inject controlled increases in fraud-labeled transaction density into replay data.

Synthetic stress tests must be reported separately from the untouched natural held-out evaluation and must never replace it.

15. Alert Decision and Severity

15.1 Alert trigger

A spike alert is generated when the selected spike score exceeds a validation-tuned threshold.

The threshold must not be tuned on the final test set.

15.2 Severity

The dashboard may expose:

- **NORMAL**
- **ELEVATED**
- **HIGH / SPIKE DETECTED**

Severity thresholds are derived from validation behavior and cost trade-offs.

15.3 No automatic fraud verdict

A spike alert means:

“Suspicious activity is materially above the expected baseline and requires defensive attention.”

It does not mean:

“Every transaction in this period is fraudulent.”

16. Cost Framework

16.1 Principle

The system must not optimize only for maximum detection.

It must balance:

- fraud losses avoided;
- fraud losses missed;
- analyst/review cost;
- customer-friction cost;
- spike-alert operational cost.

16.2 Why values remain parameterized

No real merchant has provided authoritative costs for:

- analyst time,
- customer verification friction,
- chargeback overhead,
- recovery rate,
- loss given fraud.

Therefore, the project must **not fabricate merchant-specific rupee values**.

Instead, the technical specification fixes the cost equations and evaluates them under explicit low/base/high scenarios.

16.3 Transaction-level cost model

Define:

- c_{review} = operational cost of reviewing a flagged transaction
- c_{friction} = modeled cost of inconveniencing a legitimate customer when intervention occurs
- LGF = loss-given-fraud fraction
- $c_{\text{fraud_ops}}$ = additional operational cost associated with a missed fraud case
- A_i = transaction amount for transaction i

For a false positive:

$$\text{Cost_FP_i} = c_{\text{review}} + c_{\text{friction}}$$

For a missed fraudulent transaction:

$$\text{Cost_FN_i} = (A_i * \text{LGF}) + c_{\text{fraud_ops}}$$

For a detected fraudulent transaction, the prototype may model avoided loss as:

$$\begin{aligned} \text{AvoidedLoss_TP_i} = \\ (A_i * \text{LGF} * \text{intervention_effectiveness}) - c_{\text{review}} \end{aligned}$$

The intervention-effectiveness parameter must be explicitly stated.

16.4 Spike-alert operational cost

Define:

- $c_{\text{alert_review}}$ = cost for risk-team investigation of one spike alert

Then:

$$\begin{aligned} \text{FalseSpikeAlertCost} = \\ \text{number_of_false_spike_alerts} * c_{\text{alert_review}} \end{aligned}$$

If an alert causes transaction-level reviews, those costs must be added rather than hidden.

16.5 Net modeled value

A generic net-benefit form is:

$$\begin{aligned} \text{NetBenefit} = \\ &\text{AvoidedFraudLoss} \\ &- \text{FalsePositiveCost} \\ &- \text{FraudReviewCost} \\ &- \text{FalseSpikeAlertCost} \\ &- \text{ResidualMissedFraudLoss} \end{aligned}$$

16.6 Sensitivity analysis

The final report must evaluate at least:

- low-cost scenario
- base scenario
- high-cost scenario

The project should show whether the preferred threshold/model changes as the assumed economics change.

17. Evaluation Framework

Evaluation is separated into transaction-level and spike-level performance.

17.1 Transaction-level priorities

Primary metrics:

1. **Precision**
2. **Recall**
3. **PR-AUC**

Secondary/supporting:

- confusion matrix
- false-positive rate
- ROC-AUC
- alert volume
- cost at chosen threshold

Accuracy must not be used as the headline metric.

17.2 Spike-level priorities

Primary metrics:

1. **Spike-event precision**
2. **Spike-event recall**
3. **Detection latency / time-to-detection**
4. **False-alert rate**
5. **Modeled business cost**

Supporting metrics:

- alert count
- fraction of high-loss spike periods detected
- distribution of detection delay
- window-level precision/recall as a secondary view

17.3 Event-level detection rule

A proxy true-spike event is counted as detected if the system produces a spike alert during the event or within the permitted detection tolerance defined on validation.

The final report must state the exact tolerance.

17.4 Detection latency

For each detected event:

latency =

`first_alert_time - event_onset_time`

Latency should be reported in relative minutes/hours.

17.5 Final held-out reporting

The final test period is evaluated once the model, thresholds, preprocessing, baseline parameters, and alert policy are fixed.

The report must clearly distinguish:

- tuning/validation results
 - final held-out results
-

18. Near-Real-Time Replay Architecture

18.1 Prototype deployment mode

The prototype uses **chronological replay** of historical transactions.

Historical records are processed sequentially as if they were arriving live.

18.2 Why replay instead of batch-only analysis

A fraud-spike detector is valuable only if it can detect abnormal activity before the event is long over.

Replay enables evaluation of:

- detection delay
- rolling baselines
- live-style feature computation
- alert timing
- data leakage

without requiring access to a production payment stream.

18.3 Future production mode

A production implementation would replace the replay source with an actual event stream while preserving the same logical stages:

transaction event

-> preprocessing

-> transaction risk

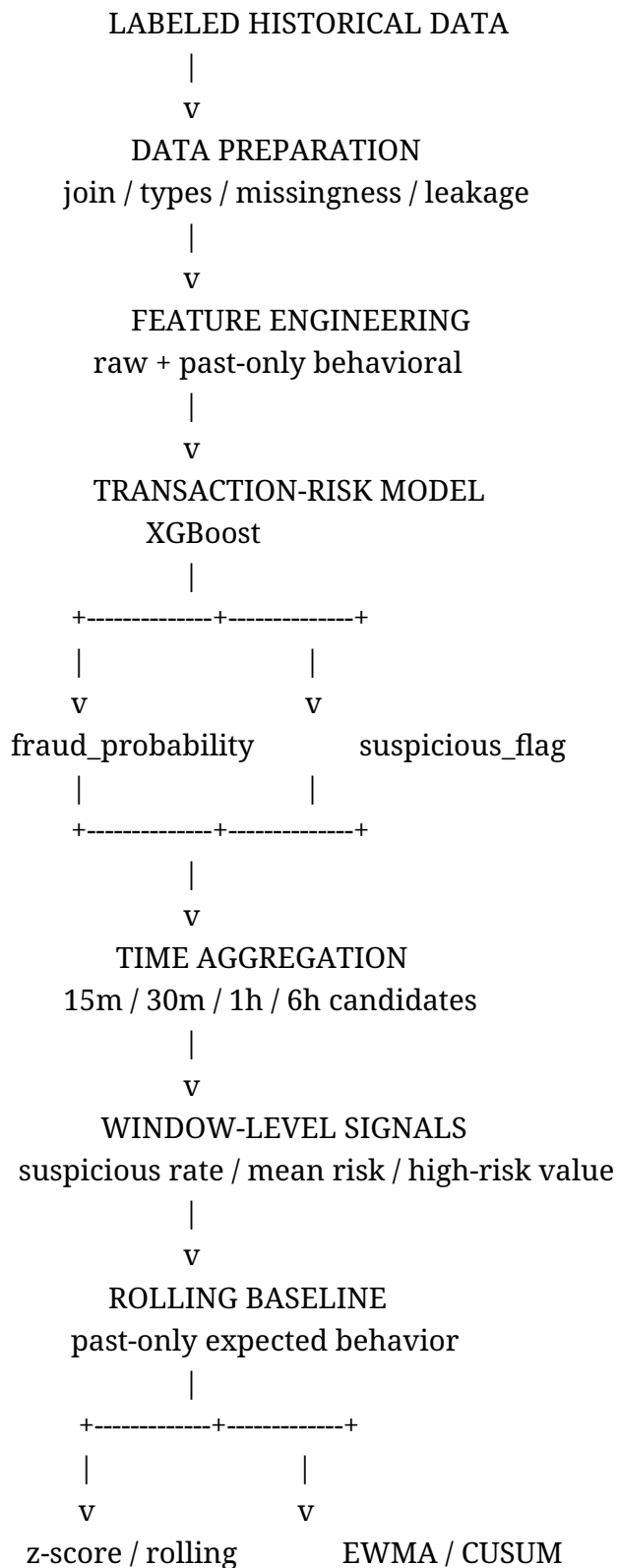
-> time aggregation

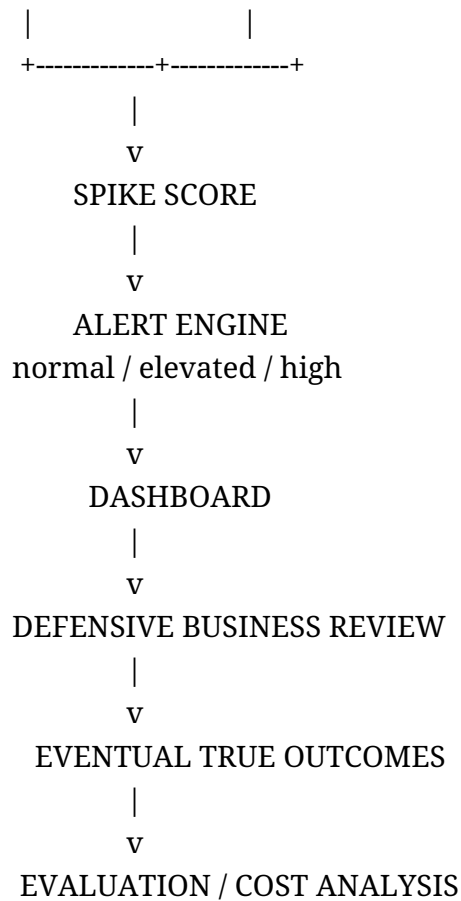
-> baseline/spike detection

-> alert

Production streaming technology is not locked in v1.

19. End-to-End System Architecture





Isolation Forest may be evaluated as an additional aggregated-feature comparator.

20. Dashboard and Alerting Requirements

The UI must answer:

1. **Is activity normal right now?**
2. **How far is current behavior from the expected baseline?**
3. **What transactions/signals are contributing to the alert?**
4. **How severe is the event?**
5. **When did it begin?**

20.1 Required dashboard elements

Current status

NORMAL / ELEVATED / SPIKE DETECTED

Suspicious-activity timeline

Plot:

- suspicious rate or selected spike signal
- rolling baseline
- alert threshold

- detected event markers

Baseline vs current

Display:

- current suspicious rate
- expected baseline
- absolute deviation
- relative deviation
- spike score

Window summary

Display:

- relative time window
- total transactions
- high-risk transactions
- mean fraud probability
- high-risk transaction amount

Alert panel

Display:

- severity
- event onset
- detection time
- spike score
- reason for alert
- recommended defensive action: investigate / increase verification / review contributing transactions

Contributing factors

Where supported by validated features, show top contributors such as:

- product/card group
- device group
- address group
- transaction-amount pattern
- risk-contributing features

The UI must not imply causal certainty when it is only displaying model associations.

20.2 Alert payload

A structured alert should contain at minimum:

alert_id
 window_start_relative
 window_end_relative
 severity
 current_signal
 expected_baseline

deviation
 spike_score
 transaction_count
 high_risk_count
 high_risk_amount
 model_version

Optional explainability fields may be added only when technically supported.

21. Modeling Experiments

The project should compare models in a controlled sequence.

Experiment A — simple transaction baseline

- simple classifier where feasible
- no advanced behavioral features

Purpose: establish a minimum supervised benchmark.

Experiment B — XGBoost raw/clean features

Purpose: establish the primary transaction-risk benchmark.

Experiment C — XGBoost + past-only behavioral features

Purpose: quantify value added by temporal/entity feature engineering.

Experiment D — rolling z-score spike detector

Purpose: transparent primary spike baseline.

Experiment E — EWMA/CUSUM

Purpose: test whether shift-detection methods improve event recall/latency.

Experiment F — Isolation Forest on window features

Purpose: compare unsupervised aggregated anomaly detection.

Each experiment must use the same chronological evaluation discipline.

22. Threshold Selection

There are at least two separate thresholds:

22.1 Transaction threshold τ_{tx}

Converts fraud probability into a suspicious flag.

Selected on validation using:

- precision/recall
- alert volume
- transaction-level cost

22.2 Spike threshold `tau_spike`

Converts the spike score into an alert.

Selected on validation using:

- spike-event precision/recall
- detection latency
- false-alert rate
- spike-alert cost

Neither threshold may be optimized on the held-out test period.

23. Reproducibility Requirements

The project must preserve:

- data version/source
- feature list
- preprocessing configuration
- train/validation/test boundary values
- random seeds where algorithms are stochastic
- XGBoost hyperparameters
- threshold values
- baseline window length
- EWMA/CUSUM parameters
- cost parameters and scenario assumptions
- model version used by each alert

All final metrics must be reproducible from a clean run.

24. Acceptance Criteria

A v1 implementation is considered complete only if all of the following are true:

- ☐ IEEE-CIS labeled data is loaded and transaction/identity data is joined correctly.
- ☐ Data is sorted by `TransactionDT`.
- ☐ Train/validation/test partitions are chronological.
- ☐ No random outer split is used.
- ☐ Missing-value transformations are fitted using training data only.
- ☐ `isFraud` is never used as an input feature.
- ☐ `TransactionID` is not used as a predictive feature.
- ☐ Time/entity behavioral features are past-only.
- ☐ XGBoost produces transaction fraud probabilities.

- ☐ A validation-selected `tau_tx` defines suspicious transactions.
 - ☐ Risk signals are aggregated in relative-time windows.
 - ☐ A rolling baseline is constructed from prior windows only.
 - ☐ At least one transparent spike detector is implemented.
 - ☐ EWMA and/or CUSUM is compared against the baseline.
 - ☐ Spike alerts are generated during chronological replay.
 - ☐ Proxy true-spike events are defined offline using actual labels.
 - ☐ The final held-out test period is not used for tuning.
 - ☐ Transaction precision/recall/PR-AUC are reported.
 - ☐ Spike precision/recall/latency/false-alert rate are reported.
 - ☐ False-positive and missed-fraud costs are explicitly modeled.
 - ☐ Cost sensitivity is reported rather than hidden behind one invented rupee assumption.
 - ☐ Dashboard clearly distinguishes risk/anomaly from confirmed fraud.
 - ☐ The project remains defense-only.
-

25. Key Risks and Limitations

25.1 No native incident-level spike labels

The biggest evaluation limitation is that IEEE-CIS labels individual fraudulent transactions, not confirmed fraud-campaign incidents.

The project therefore uses a transparent proxy spike definition and sensitivity analysis.

25.2 Relative time, not real timestamp

The project can simulate chronological detection and fixed-duration windows, but it cannot truthfully infer real local clock times from `TransactionDT` alone.

25.3 Anonymized features

Many variables have masked semantics, limiting business interpretability.

25.4 Missing identity rows

Not every transaction has identity/device information.

25.5 Cost assumptions

Without a merchant partner, monetary costs remain scenario parameters.

25.6 Dataset-to-production gap

Offline replay is not equivalent to production streaming. The v1 prototype validates logic and timing discipline, not production throughput guarantees.

26. Locked Decisions vs Data-Driven Decisions

26.1 Locked

- Defense-only fraud-spike project.
- IEEE-CIS labeled training data as primary dataset.
- Supervised transaction-risk layer.
- XGBoost as primary transaction classifier.
- Chronological ordering by **TransactionDT**.
- 70% / 15% / 15% chronological outer split.
- Expanding-window CV inside training.
- Train-only preprocessing.
- Median numerical imputation.
- Explicit **Unknown** categorical missing value.
- Missingness indicators where validated.
- Past-only behavioral features.
- Transaction probability + suspicious flag retained.
- Relative 1-hour window as initial aggregation choice.
- Rolling baseline as primary baseline.
- z-score/rolling-statistic detector as transparent spike baseline.
- EWMA/CUSUM comparison.
- Isolation Forest as optional experimental comparator.
- Near-real-time chronological replay.
- Cost-aware thresholding.
- Held-out evaluation.
- Merchant-facing alert/dashboard.
- No automatic fraud verdict.

26.2 Selected on validation / EDA

- Exact final raw feature list.
- Which high-missingness features survive.
- Which missingness indicators survive.
- Encoding strategy for individual categorical features.
- XGBoost hyperparameters.
- final time-window size.
- **tau_tx**.
- rolling-history length.
- spike-score parameters.
- **tau_spike**.
- EWMA/CUSUM parameters.
- minimum eligible transaction volume for proxy spike labels.
- event detection tolerance.
- alert severity boundaries.
- low/base/high cost parameter values.

This separation is intentional: these values cannot be honestly “locked” before observing training/validation behavior.

27. Final System Definition

The AI Fraud-Spike Detector is a defense-only, two-layer fraud-risk monitoring system. The first layer uses historical confirmed transaction outcomes to train an XGBoost-based transaction-risk model that produces fraud probabilities for chronologically arriving transactions. The second layer aggregates those risk signals across relative-time windows, compares current behavior against a past-only rolling baseline, and uses statistical/change-detection methods to identify meaningful increases in suspicious activity. The system is evaluated through chronological out-of-time replay using a held-out test period, with separate transaction-level and spike-event metrics, explicit detection-latency measurement, and a parameterized financial/operational cost framework. Alerts indicate elevated risk requiring defensive investigation; they do not constitute confirmed fraud verdicts.

28. Implementation Sequence

Phase 1 — data audit

1. Acquire IEEE-CIS labeled training files.
2. Join transaction and identity tables.
3. Verify row counts, uniqueness, time ordering, target distribution, and missingness.
4. Produce EDA report.
5. Freeze chronological split boundaries.

Phase 2 — leakage-safe preprocessing

1. Build train-only imputation pipeline.
2. Build categorical handling.
3. Implement past-only temporal/entity features.
4. Add missingness indicators where validated.
5. Log all feature definitions.

Phase 3 — transaction-risk model

1. Train baseline classifier.
2. Train XGBoost under expanding-window CV.
3. Tune hyperparameters.
4. Fit on full 70% training period.
5. Use 15% validation period to select `tau_tx`.

Phase 4 — aggregation and spike detection

1. Replay validation transactions chronologically.
2. Aggregate transaction-risk signals.
3. Build rolling baseline.
4. Fit/tune rolling z-score rule.
5. Compare EWMA/CUSUM.
6. Optionally evaluate Isolation Forest.
7. Freeze spike parameters.

Phase 5 — cost and alert policy

1. Define low/base/high cost scenarios.
2. Select operating thresholds.
3. Define NORMAL/ELEVATED/HIGH states.

4. Freeze alert payload and UI fields.

Phase 6 — final held-out replay

1. Refit the permitted final model using only allowed pre-test data according to the frozen protocol.
 2. Replay the latest 15% in chronological order.
 3. Generate alerts without using future labels.
 4. Reveal labels offline.
 5. Compute transaction and spike metrics.
 6. Compute cost sensitivity.
 7. Produce final dashboard/report.
-

29. Dataset Verification References

The dataset facts in this specification were rechecked against current public descriptions of the IEEE-CIS competition data:

1. Kaggle — IEEE-CIS Fraud Detection data page
<https://www.kaggle.com/competitions/ieee-fraud-detection/data>
 2. Public IEEE-CIS dataset overview mirror describing transaction/identity feature groups and TransactionDT
https://github.com/prathik-k/IEEE-CIS_Fraud-Detection/blob/gh-pages/Dataset%20Overview.md
 3. Public competition-description mirror noting that transaction and identity data join on TransactionID, not every transaction has identity data, and TransactionDT is a time delta rather than a timestamp
<https://github.com/Zheyuan-Chen/IEEE-CIS-Fraud-Detection/blob/master/README.md>
-

30. Version Notes

v1.0

Compiled from the finalized project discussions and rechecked for internal consistency.

The recheck made three important clarifications:

1. **Relative time:** the “1-hour” aggregation window is 3,600 seconds of TransactionDT, not a verified wall-clock hour.
2. **Spike ground truth:** IEEE-CIS does not provide native spike labels; the specification therefore defines a transparent offline proxy-labeling protocol plus sensitivity analysis.
3. **Cost values:** the cost equations are fixed, but merchant-specific rupee values remain explicit parameters until real operational data is available.

These clarifications prevent temporal leakage, false claims about the dataset, and fabricated business economics.